

LAPORAN PRAKTIKUM

APLIKASI BERBASIS PLATFORM

Modul 12 - 13

IMPLEMENTASI PROVIDER DAN NOTIFIKASI
PADA FLUTTER



Disusun oleh:

Muhammad Daffa Bagus Jumanoro

2311102222

S1 IF-11-04

Dosen Pengampu:

Cahyo Prihantoro, S.Kom., M.Eng

PROGRAM STUDI S1 INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

Daftar Isi

1. [Deskripsi Program](#)
 2. [Dependencies yang Digunakan](#)
 3. [Struktur Program](#)
 4. [Penjelasan Source Code](#)
 - [1. File pubspec.yaml](#)
 - [2. main.dart](#)
 - [3. CounterProvider \(counter_provider.dart\)](#)
 - [4. NotificationService \(notification_service.dart\)](#)
 - [5. Android Build Configuration \(build.gradle.kts\)](#)
 - [6. AndroidManifest.xml](#)
 5. [Cara Kerja Provider pada Aplikasi](#)
 6. [Cara Kerja Notifikasi pada Aplikasi](#)
 7. [Output Program](#)
-

Deskripsi Program

Program ini merupakan aplikasi Flutter bertema **Coffee Counter** yang menerapkan konsep **State Management menggunakan Provider** dan **Notifikasi Lokal menggunakan flutter_local_notifications**.

Aplikasi ini menampilkan:

- **Kartu identitas** mahasiswa dengan nama dan NIM
- **Counter digital** yang dapat bertambah saat tombol ditekan
- **Notifikasi lokal** yang muncul setiap kali nilai counter bertambah

Program dirancang dengan tema warna coklat (brown) yang merepresentasikan tema coffee, dilengkapi dengan ikon kopi pada header.

Dependencies yang Digunakan

Pada project ini digunakan beberapa package tambahan yang terdapat pada file `pubspec.yaml`.

1. provider

```
provider: ^6.1.2
```

Fungsi

Package **Provider** digunakan sebagai state management untuk mengelola dan membagikan data (state) antar widget dalam aplikasi Flutter.

Import

```
import 'package:provider/provider.dart';
```

Implementasi

```
ChangeNotifierProvider(  
  create: (_) => CounterProvider(),  
  child: MaterialApp(...),  
)
```

Cara Kerja

Provider bekerja dengan menyediakan class `ChangeNotifier` yang dapat memberitahu widget lain ketika terjadi perubahan data. Widget yang ingin mengakses data dapat menggunakan `context.watch<T>()` untuk membaca data atau `context.read<T>()` untuk melakukan aksi tanpa mendengarkan perubahan.

2. flutter_local_notifications

```
flutter_local_notifications: ^19.4.0
```

Fungsi

Package ini digunakan untuk menampilkan notifikasi lokal pada perangkat Android.

Import

```
import 'package:flutter_local_notifications/flutter'
```

Implementasi

```
final FlutterLocalNotificationsPlugin notificationPlugin =  
FlutterLocalNotificationsPlugin();
```

Struktur Program

```
lib  
|  
├── providers  
|   └── counter_provider.dart  
|  
├── services  
|   └── notification_service.dart  
|  
└── main.dart
```

Penjelasan Struktur

File / Folder	Fungsi
<code>lib/main.dart</code>	Entry point aplikasi, konfigurasi Provider & MaterialApp
<code>lib/providers/</code>	Folder berisi class state management
<code>counter_provider.dart</code>	Class pengelola state counter menggunakan ChangeNotifier
<code>lib/services/</code>	Folder berisi service / layanan aplikasi
<code>notification_service.dart</code>	Service untuk menampilkan notifikasi lokal

Penjelasan Source Code

1. File pubspec.yaml

```
dependencies:  
  flutter:  
    sdk: flutter  
  cupertino_icons: ^1.0.8  
  provider: ^6.1.2  
  flutter_local_notifications: ^19.4.0
```

Penjelasan:

File `pubspec.yaml` digunakan untuk menentukan dependencies atau package yang dibutuhkan oleh aplikasi. Pada project ini ditambahkan dua package utama:

- **provider**: Digunakan untuk state management.
- **flutter_local_notifications**: Digunakan untuk menampilkan notifikasi lokal.

Setelah menambahkan dependencies, jalankan perintah `flutter pub get` untuk mengunduh package tersebut.

2. main.dart

Import Library

```
import 'package:flutter/material.dart';  
import 'package:provider/provider.dart';  
import 'providers/counter_provider.dart';  
import 'services/notification_service.dart';
```

Penjelasan:

- `material.dart` digunakan untuk mengakses widget dasar Flutter.
 - `provider.dart` digunakan untuk mengakses fitur state management Provider.
 - `counter_provider.dart` adalah class yang mengelola state counter.
 - `notification_service.dart` adalah service untuk menampilkan notifikasi.
-

Fungsi main()

```
void main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  await NotificationService().init();  
  runApp(const MyApp());  
}
```

Penjelasan:

Fungsi `main()` merupakan entry point aplikasi.

- `WidgetsFlutterBinding.ensureInitialized()` digunakan untuk memastikan binding Flutter telah diinisialisasi sebelum menjalankan kode async.
 - `NotificationService().init()` digunakan untuk menginisialisasi plugin notifikasi dan meminta izin notifikasi pada perangkat Android.
 - `runApp(const MyApp())` digunakan untuk menjalankan aplikasi Flutter.
-

Widget MyApp (Konfigurasi Provider dan MaterialApp)

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return ChangeNotifierProvider(  
      create: (_) => CounterProvider(),  
      child: MaterialApp(  
        debugShowCheckedModeBanner: false,  
        title: 'Coffee Counter',  
        theme: ThemeData(  
          colorScheme: ColorScheme.fromSeed(seedColor: Colors.brown),  
          useMaterial3: true,  
        ),  
        home: const HomePage(),  
      ),  
    );  
  }  
}
```

```
}  
}
```

Penjelasan:

- `ChangeNotifierProvider` adalah widget Provider yang menyediakan instance `CounterProvider` ke seluruh widget di dalamnya.
 - `create: (_) => CounterProvider()` digunakan untuk membuat instance baru `CounterProvider`.
 - `MaterialApp` adalah root widget aplikasi dengan tema warna coklat (brown) dan `Material3` diaktifkan.
 - `home: const HomePage()` menentukan halaman utama aplikasi.
-

Widget HomePage

```
class HomePage extends StatelessWidget {  
  const HomePage({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    final counterProvider = context.watch<CounterProvider>();  
  
    return Scaffold(  
      backgroundColor: const Color(0xffff5f0eb),  
      appBar: AppBar(  
        title: const Text(  
          'Modul 12-13',  
          style: TextStyle(fontWeight: FontWeight.bold),  
        ),  
        centerTitle: true,  
        backgroundColor: Colors.brown,  
        foregroundColor: Colors.white,  
      ),  
      body: Center(  
        child: Column(  
          mainAxisAlignment: MainAxisAlignment.center,  
          children: [  
            // Identity Card  
            Container(  
              width: 200, height: 100, color: Colors.white,  
              margin: 10, border: Border.all(color: Colors.brown, width: 2),  
            ),  
          ],  
        ),  
      ),  
    );  
  }  
}
```

```

margin: const EdgeInsets.symmetric(
padding: const EdgeInsets.all(24),
decoration: BoxDecoration(
  gradient: LinearGradient(
    colors: [Colors.brown, Colors.b:
  ),
  borderRadius: BorderRadius.circular
),
child: const Column(
  children: [
    Icon(Icons.coffee, color: Colors:
    SizedBox(height: 8),
    Text(
      "Muhammad Daffa Bagus Jumanto:
      style: TextStyle(
        color: Colors.white,
        fontSize: 20,
        fontWeight: FontWeight.bold
      ),
    ),
    SizedBox(height: 4),
    Text(
      "2311102222",
      style: TextStyle(
        color: Colors.white70,
        fontSize: 16,
      ),
    ),
  ],
),
),

const SizedBox(height: 40),

// Counter Display
Text(
  'Counter Saat Ini',
  style: TextStyle(
    fontSize: 18,
    color: Colors.brown.shade600,
    fontWeight: FontWeight.w500,
  ),
),
const SizedBox(height: 8),

```



```

        Text(
          '${counterProvider.counter}',
          style: TextStyle(
            fontSize: 72,
            fontWeight: FontWeight.bold,
            color: Colors.brown.shade800,
          ),
        ),

        const SizedBox(height: 32),

        // Increment Button
        ElevatedButton.icon(
          onPressed: () {
            counterProvider.increment();
            NotificationService().showNotification(
              title: "Counter Update",
              body: "Nilai counter saat ini: ${counterProvider.counter}",
            );
          },
          icon: const Icon(Icons.add),
          label: const Text("Tambah +1"),
          style: ElevatedButton.styleFrom(
            backgroundColor: Colors.brown,
            foregroundColor: Colors.white,
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(10),
            ),
          ),
        ),
      ],
    ),
  ),
);
}
}

```

Penjelasan:

- `context.watch<CounterProvider>()` digunakan untuk membaca nilai counter dari Provider. Setiap kali nilai berubah, widget akan di-rebuild secara otomatis.

- **Identity Card:** Container dengan gradasi warna coklat yang menampilkan ikon kopi, nama, dan NIM.
 - **Counter Display:** Menampilkan nilai counter saat ini dengan ukuran font besar (72).
 - **Tombol Tambah +1:** Ketika ditekan, akan memanggil `counterProvider.increment()` untuk menambah counter dan `NotificationService().showNotification()` untuk menampilkan notifikasi.
-

3. CounterProvider (counter_provider.dart)

```
import 'package:flutter/foundation.dart';

class CounterProvider extends ChangeNotifier {
  int _counter = 0;

  int get counter => _counter;

  void increment() {
    _counter++;
    notifyListeners();
  }
}
```

Penjelasan:

Class `CounterProvider` merupakan class state management yang mewarisi `ChangeNotifier`.

- `_counter` adalah variabel private yang menyimpan nilai counter.
 - `get counter` adalah getter untuk mengakses nilai counter dari luar class.
 - `increment()` adalah method untuk menambah nilai counter. Setelah nilai berubah, method `notifyListeners()` dipanggil untuk memberi tahu seluruh widget yang sedang "mendengarkan" (watching) Provider bahwa terjadi perubahan data.
-

4. NotificationService (notification_service.dart)

```

import 'package:flutter_local_notifications/flutter_local_notifications.dart';

class NotificationService {
  static final NotificationService _instance = NotificationService();
  factory NotificationService() => _instance;
  NotificationService._internal();

  final FlutterLocalNotificationsPlugin _notifications =
    FlutterLocalNotificationsPlugin();

  Future<void> init() async {
    const AndroidInitializationSettings androidSettings =
      AndroidInitializationSettings('@mipmap/ic_launcher');

    const InitializationSettings settings =
      InitializationSettings(android: androidSettings,
        ios: null);

    await _notifications.initialize(settings);

    final androidPlugin =
      _notifications.resolvePlatformSpecificImplementation(
        AndroidFlutterLocalNotificationsPlugin());

    await androidPlugin?.requestNotificationsPermissions();
  }

  Future<void> showNotification({
    required String title,
    required String body,
  }) async {
    const AndroidNotificationDetails androidDetails =
      AndroidNotificationDetails(
        'coffee_counter_channel',
        'Coffee Counter',
        channelDescription: 'Notifikasi Counter Coffee',
        importance: Importance.max,
        priority: Priority.high,
      );

    const NotificationDetails details =
      NotificationDetails(android: androidDetails,
        ios: null);

    await _notifications.show(
      0,

```

```

        title,
        body,
        details,
    );
}
}

```

Penjelasan:

`NotificationService` merupakan **Singleton class** yang menangani semua operasi notifikasi lokal.

- `_instance` : Instance tunggal dari `NotificationService` (Singleton pattern).
- `init()` : Method untuk menginisialisasi plugin notifikasi dan meminta izin notifikasi pada Android.
- `showNotification()` : Method untuk menampilkan notifikasi dengan parameter title dan body.
 - `AndroidNotificationDetails` digunakan untuk mengkonfigurasi tampilan notifikasi seperti channel ID, importance, dan priority.

5. Android Build Configuration (build.gradle.kts)

```

compileOptions {
    isCoreLibraryDesugaringEnabled = true
}

dependencies {
    coreLibraryDesugaring("com.android.tools:desugaring")
}

```

Penjelasan:

Konfigurasi ini diperlukan karena package

`flutter_local_notifications` membutuhkan **core library desugaring** untuk menjalankan fitur tertentu pada perangkat Android.

- `isCoreLibraryDesugaringEnabled = true` mengaktifkan desugaring.

- `coreLibraryDesugaring("com.android.tools:desugar_jdk_libs:2.1.4")` menambahkan dependency library desugaring.
-

6. AndroidManifest.xml

```
<uses-permission android:name="android.permission
```

Penjelasan:

Permission `POST_NOTIFICATIONS` diperlukan pada Android 13+ (API level 33+) untuk dapat menampilkan notifikasi.

Cara Kerja Provider pada Aplikasi

Pada aplikasi ini digunakan package **Provider** sebagai state management untuk mengelola nilai counter. Provider bekerja dengan membuat class `CounterProvider` yang mewarisi `ChangeNotifier`. Class tersebut menyimpan nilai counter dan menyediakan method `increment()` untuk menambah nilai counter.

Alur kerja:

1. `MyApp` membungkus seluruh aplikasi dengan `ChangeNotifierProvider` yang menyediakan instance `CounterProvider`.
2. `HomePage` menggunakan `context.watch<CounterProvider>()` untuk mendengarkan perubahan nilai counter.
3. Ketika tombol **Tambah +1** ditekan, method `increment()` pada `CounterProvider` akan dipanggil.
4. Di dalam `increment()`, nilai `_counter` bertambah satu, lalu `notifyListeners()` dipanggil.
5. `notifyListeners()` memberi tahu seluruh widget yang menggunakan `context.watch<CounterProvider>()` bahwa data telah berubah.
6. Widget akan di-rebuild secara otomatis, menampilkan nilai counter terbaru.

Dengan Provider, perubahan data dapat dikelola secara efisien tanpa perlu melakukan refresh manual pada halaman.

Cara Kerja Notifikasi pada Aplikasi

Pada aplikasi ini digunakan package **flutter_local_notifications** untuk menampilkan notifikasi lokal pada perangkat Android.

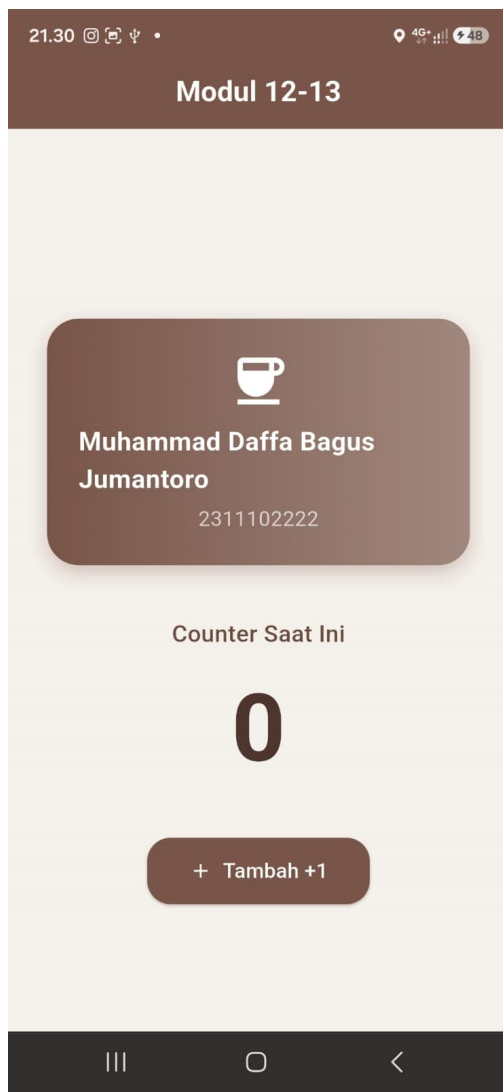
Alur kerja:

1. **Inisialisasi:** Saat aplikasi dijalankan, `NotificationService().init()` dipanggil di fungsi `main()` untuk menginisialisasi plugin notifikasi dan meminta izin notifikasi.
2. **Trigger Notifikasi:** Setiap kali tombol **Tambah +1** ditekan, setelah counter diincrement, aplikasi memanggil `NotificationService().showNotification()`.
3. **Parameter Notifikasi:**
 - **title:** "Counter Update"
 - **body:** "Nilai counter saat ini: X" (X adalah nilai counter terbaru)
4. **Tampilan Notifikasi:** Notifikasi akan muncul pada perangkat Android dengan judul dan body yang telah ditentukan.

Dengan demikian, setiap perubahan nilai counter akan langsung memberikan informasi kepada pengguna melalui notifikasi lokal yang muncul pada perangkat.

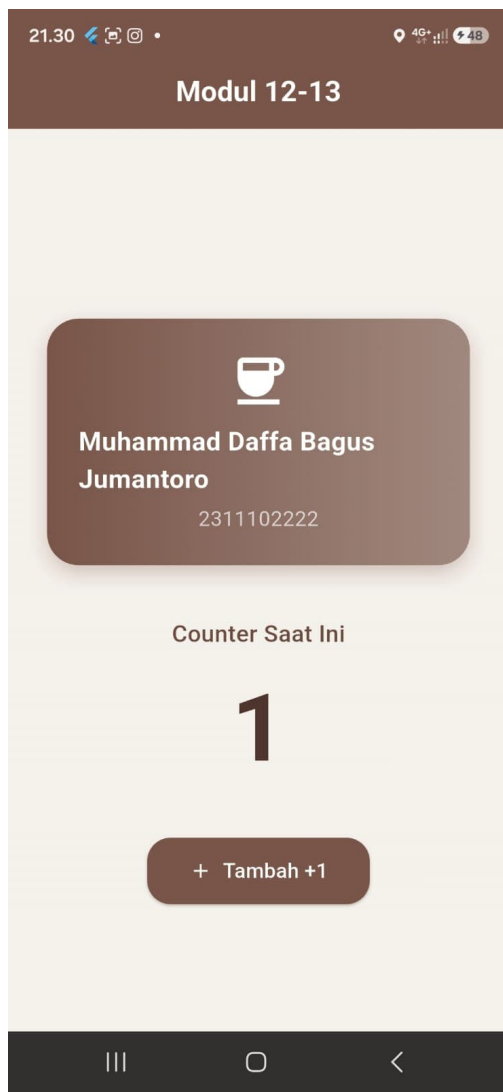
Output Program

Output 1 - Tampilan Awal Aplikasi



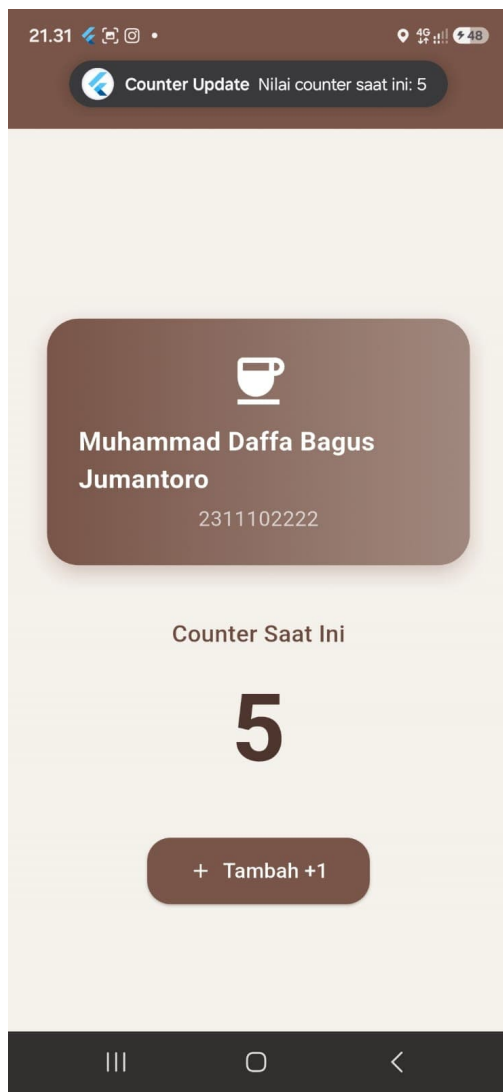
Tampilan awal aplikasi Coffee Counter saat pertama kali dijalankan.

Output 2 - Setelah Tombol Tambah Ditekan



Tampilan aplikasi setelah tombol **Tambah +1** ditekan beberapa kali.

Output 3 - Notifikasi Lokal



Notifikasi lokal yang muncul setiap kali tombol **Tambah +1** ditekan, menampilkan pesan "Nilai counter saat ini: X".